Bacha Hua Khana
V2 features and how to test them

Covers PROMPTS_V2 features 1–8 · state of the code on 27 Oct 2026 · 534 automated tests passing

What each V2 feature added, and three ways to check it: the automated tests (free, mock mode), a hands-on walk-through in the browser (free, mock mode), and a short live check with real Gnani speech (spends credits, so do these last and on purpose).

ONE-TIME SETUP FOR EVERY CHECK

```
cd C:\Users\vaibh\Desktop\bacha-hua-khana\backend
.venv\Scripts\Activate.ps1
python -m pytest -q                                  # all 534 tests, about 40 s
$env:MOCK_GNANI="1"; uvicorn app.main:app --port 8000
```

Open `localhost:8000/kitchen/`, `/volunteer/`, `/ngo/` and `/dashboard/` in separate tabs. In mock mode each page has a yellow box: type what you would say, then press **Send without mic**. An empty box sends a canned "yes" or a sample sentence.

> **Use real clock times.** Food must be picked up within 4 hours of cooking (`SAFE_WINDOW_HOURS`). In the sample sentences, replace *aath baje / 8 pm* with a time from the last hour or two, or the report is (correctly) marked EXPIRED.

> **Start clean.** Press **Reset demo data** on the dashboard before each walk-through. It clears all reports and opens one need: Karunya Children's Home, 80 people, about 3 hours ahead.

THE EIGHT FEATURES AT A GLANCE

#	Feature	What you can now do	Automated tests
1	Telephony eval mode	Measure accuracy on 8 kHz phone-line audio; run the app in phone-line mode	test_audio, test_eval (36)
2	Malayalam and Manglish	Report and confirm in Malayalam script or Manglish (plus Kannada); language picker per kitchen	test_languages, test_manglish, test_extractor, test_confirm (177)
3	NGO demand side	Shelters ask for food by voice; food is routed kitchen → volunteer → NGO; NGO confirms delivery	test_needs, test_delivery, test_matcher (71)
4	Impact and share card	Live impact numbers; a 1080×1080 card per finished rescue	test_impact, test_cards (32)
5	Voice status check	Kitchen asks "volunteer kab aayega?" or cancels by voice	test_ask (59)
6	Spoken safety check	"Veg?" and "covered?" before any matching; uncovered food never sent	test_safety (41)
7	"Kal jaisa"	Repeat yesterday's report with one phrase, plus changes	test_repeat (56)
8	Integration pass	Clean demo reset with a need and a planned fallback; TTS cache check; bug fixes	test_dashboard, test_tts_warm, test_cards (49)

FEATURE 1

Telephony eval mode

> Prove the app understands narrow-band phone-call audio, not just clean microphone audio.

WHAT WAS ADDED

- `audio.to_telephony()`: 8 kHz mono, 300–3400 Hz band-pass, light compression and a G.711 μ-law round trip, so a clip sounds like a phone call. The normal 16 kHz path is unchanged.
- `TELEPHONY_MODE=1`: the live app passes every recording through that filter before Prisma. The dashboard then shows a 📞 **Phone line mode · 8 kHz** badge.
- `eval/run_eval.py --mode clean|telephony|all`: transcripts are cached per mode (`eval/transcripts_<mode>.json`), so reruns cost nothing. Clips can be tagged quiet or noisy.
- A quiet / noisy / telephony comparison table, written to `eval/RESULTS.md`. Latest result: 16/18 clips fully right in all three conditions. Both misses are speech-to-text errors; the extractor gets 18/18 on the exact words.

AUTOMATED TESTS

```
python -m pytest -q tests/test_audio.py tests/test_eval.py
```

Checks: telephony output is 8 kHz mono; cached transcripts are reused with no API call; the table and RESULTS.md are built correctly.

TRY IT (FREE)

1. From the project root, re-score every mode from cached transcripts:

   ```
   backend\.venv\Scripts\python eval\run_eval.py
   ```

   → the comparison table prints and eval/RESULTS.md is rewritten, with no credits spent.

2. Restart the server with the badge on:

   ```
   $env:MOCK_GNANI="1"; $env:TELEPHONY_MODE="1"; uvicorn app.main:app --port 8000
   ```

   → the dashboard header shows "📞 Phone line mode · 8 kHz". (Mock mode skips audio conversion, so this checks the badge only.)

LIVE CHECK (SPENDS CREDITS)

- With `MOCK_GNANI=0` and `TELEPHONY_MODE=1`, report food from a real microphone. Check the dashboard timeline: the "Heard" line should still be right.
- New clips: `run_eval.py --mode all --transcribe` costs 1 STT credit per uncached clip per mode.

FEATURE 2

Full Malayalam and Manglish flow

> A Kochi kitchen reports in Malayalam or Manglish and hears the confirmation in Malayalam. Hinglish keeps working.

WHAT WAS ADDED

- **A language per kitchen** (`Kitchen.language`). The first visit on a device shows big language buttons (हिन्दी · മലയാളം · English · ಕನ್ನಡ). The choice is saved in the browser, and **Change language** brings the buttons back.
- **Automatic detection was checked:** Gnani's live STT (REST and WebSocket) requires `language_code`. Only the slow Batch API detects language, so the picker stays and there is no `LANG_AUTO`.
- **Language packs** (`app/languages/hi, ml, en, kn`): numbers, units, times, dishes, places and yes/no words, in Malayalam script and Manglish (*ambathu, uchakku rendu manikku, Vyttila il, baakki und*). Malayalam suffixes are handled (മണിക്ക് = മണി + ക്ക്).
- **Malayalam replies:** *athe, sheri, ok, aanu / illa, alla, venda, thettu*. A correction still changes only the field mentioned.
- **Speech:** confirmations are spoken in the kitchen's language. The Malayalam voice comes from `TTS_VOICE_ML` (Reshma).
- **Beyond the prompt:** a Kannada pack (Saanvi voice), and 5 Malayalam ground-truth templates in `eval/audio/real/ml-01…05.json`, ready to record.

AUTOMATED TESTS

```
python -m pytest -q tests/test_languages.py tests/test_manglish.py tests/test_extractor.py
tests/test_confirm.py
```

TRY IT (FREE)

1. Kitchen page: choose **Malabar Kitchen, Edappally**. If the language buttons show, tap **മലയാളം**.
2. Malayalam script: **50 പ്ലേറ്റ് ബിരിയാണി ബാക്കിയുണ്ട്, രാത്രി 8 മണിക്ക് ഉണ്ടാക്കിയത്, ഇടപ്പള്ളി** → the card shows ബിരിയാണി · 50 plates · 8:00 pm · Edappally, and the read-back is in Malayalam.
3. Correct it in Manglish: **alla, arupathu plate** → only the quantity changes to 60; the prompt starts "ശരി, മാറ്റി".
4. Confirm: **athe** → the safety check starts (Feature 6).
5. **Start a new report** and try Manglish: **ambathu plate biryani baakki und, ettu manikku, Edappally il** → 50 plates of biryani at 8 pm, Edappally; the read-back is still in Malayalam script.
6. Hinglish still works: choose Annapurna Mess (Hindi) and send **pachaas plate veg biryani hai, aath baje bana tha, Kakkanad se**.

LIVE CHECK (SPENDS CREDITS)

- Speak the same sentences. Prisma `ml-IN` returns Malayalam script. Check that the read-back sounds natural, ideally with a native speaker: the Malayalam and Kannada prompts were not written by native speakers.

FEATURE 3

NGO demand side

> Food goes where it is needed: kitchen (supply) → volunteer (courier) → NGO or shelter (demand).

WHAT WAS ADDED

- **New data:** the `Ngo` and `Need` models, and `Report.need_id`. Three synthetic shelters: Asha Night Shelter (Hindi), Sneha Shelter Home (Malayalam), Karunya Children's Home (English).
- **NGO page** (`/ngo/`): hold to talk, e.g. "*aaj raat 80 logon ke liye khana chahiye*". It reads people, needed-by time and veg-only, then confirms by voice like the kitchen flow.
- **Need matching:** veg-compatible (veg-only needs get only food answered "veg"), arrives before `needed_by` and the food's deadline, feeds at least half the people (`NEED_MIN_SHARE`). The nearest NGO wins. No fitting need: a plain volunteer pickup, as before.
- **Volunteers:** routes are volunteer → kitchen → NGO, and the spoken alert says where to collect **and** where to deliver.
- **Delivery:** the volunteer taps **I've delivered it**; the NGO page asks "*Khana aa gaya?*" in its language, and a "haan" moves the report and the need to **DELIVERED**.
- **Dashboard:** the chain per card (kitchen → volunteer → NGO) and a list of NGO needs.

AUTOMATED TESTS

```
python -m pytest -q tests/test_needs.py tests/test_delivery.py tests/test_matcher.py
```

TRY IT (FREE)

1. NGO page as **Asha Night Shelter**: raat 11 baje tak 40 logon ke liye sirf veg khana chahiye → read-back: 40 people, by 11 pm, veg only. Send **haan**, and the request shows as "waiting for food".
2. NGO page as **Karunya Children's Home** (it already has the reset's need). Volunteer page as **John Mathew**: tap **I'm on duty**.
3. Kitchen (Annapurna, Hindi): report 50 plates as above, confirm, and answer the safety questions with yes. → the NGO page says food is coming. John's offer names Annapurna (collect) and Karunya (deliver).
4. As John: **Haan**, then **I've picked it up**, then **I've delivered it**. → the Karunya page shows "Food arriving · Has the food arrived?".
5. Tap **Haan · Yes** on the NGO page. → "Delivered ✓"; the dashboard card moves to Delivered and shows the full chain.
6. Optional: tap **Nahi** instead. The answer is logged on the timeline and the report stays PICKED_UP.

LIVE CHECK (SPENDS CREDITS)

- Record one need by voice in each NGO language, and say the delivery "haan" aloud.

FEATURE 4

Impact summary and shareable card

> Visible impact numbers, and a shareable image for every completed rescue.

WHAT WAS ADDED

- `GET /impact`**:** today and all-time figures for meals rescued, rescues, kitchens, volunteers, NGOs and people served.
  - Plates and boxes count as one meal each.
  - Kilos and litres use `MEALS_PER_KG` (default 4), reported separately as an estimate.
  - No CO_2 or money figures.
- **Dashboard impact strip:** at the top, updating live over WebSocket.
- **Card:** `/cards/{id}.png` is a 1080×1080 card ("Annapurna Mess shared 50 meals", volunteer first name, NGO, date, "Bacha Hua Khana · built on Gnani AI"). It is made once a report is DELIVERED, or PICKED_UP when there is no NGO.
- **Share:** a **Share** button on finished dashboard cards, with **Download PNG** and the phone share sheet.
- **Fonts:** Noto Sans fonts are bundled for Latin, Devanagari, Malayalam and Kannada, with HarfBuzz text shaping, so names in any script render correctly.

AUTOMATED TESTS

```
python -m pytest -q tests/test_impact.py tests/test_cards.py
```

TRY IT (FREE)

1. Finish one rescue (Feature 3 walk-through). → the dashboard strip goes to 50 meals rescued and 50 people served.
2. Press **Share** on the Delivered card. → the card opens; **Download PNG** saves a 1080×1080 image.
3. Estimated meals: report **20 kilo chawal hai, aath baje bana tha** and finish the rescue (20 kg counts as 80 plates, so it can also go to an 80-person need). → 80 more meals, shown with ≈ because kilos are estimated.
4. Raw numbers: open `localhost:8000/impact`.

LIVE CHECK

- No credits needed. On a real phone, check that **Share…** opens the share sheet and the image posts cleanly to WhatsApp or Instagram.

FEATURE 5

Voice status check and cancel

> After confirming, the kitchen holds the mic and asks how its pickup is going, or cancels it, in its own language.

WHAT WAS ADDED

- **Intents** (`intents.py`): STATUS / CANCEL / UNKNOWN, recognised from phrases in every language ("*volunteer kab aayega?*", "*volunteer eppo varum?*", "*rehne do*", "*venda*").
- **`POST /reports/{id}/ask`:** answers from the report's real state, in the kitchen's language. Possible answers: searching, offered to a volunteer, on the way with an ETA rounded to 5 minutes, picked up, delivered.
- **Cancel** is always asked back first; only a yes cancels. It withdraws open offers (WITHDRAWN), tells an assigned volunteer and reopens the NGO's need. After pickup it is too late.
- **Wording** never assumes the volunteer's gender ("*John raaste mein hain*").

AUTOMATED TESTS

```
python -m pytest -q tests/test_ask.py
```

TRY IT (FREE)

1. Report and confirm food, and answer both safety questions with yes. The button becomes **Hold to ask: status or cancel**.
2. Ask before anyone accepts: **volunteer kab aayega?** → "...se pooch rahe hain" (asking a volunteer), or "Abhi koi volunteer free nahi hai" (nobody free yet).
3. Accept as John on the volunteer page, then ask again. → "John raaste mein hain. Pahunchne mein lagbhag 25 minute." The status line shows the ETA.
4. Cancel: **cancel kar do** → "Kya yeh pickup cancel karein?" Then **haan** → CANCELLED: John's page says the kitchen cancelled, and the NGO need opens again.
5. In Malayalam (Malabar Kitchen): **volunteer eppo varum?** The answer comes back in Malayalam.

LIVE CHECK (SPENDS CREDITS)

- Ask the question aloud in each language. ETAs are rounded to 5 minutes, so few distinct answers need TTS.

FEATURE 6

Spoken food safety check

> Make food safety concrete: two quick spoken questions after confirmation, before any matching.

WHAT WAS ADDED

- **New state: SAFETY_CHECK** between CONFIRMED and MATCHING.
- **The two questions:** Timbre asks "*Khana veg hai?*", stored as `Report.veg`, then "*Khana dhaka hua aur saaf bartan mein hai?*", stored as `Report.covered`.
- **Reading the veg answer:** "veg hai" and "non-veg hai" work without haan/nahi. Any mention of non-veg means non-veg.
- **Cover it first:** a no to the second question gets "*Kripya khana dhak dijiye, phir haan boliye*" until a yes. Uncovered food is never offered; the offer engine refuses it too.
- **EXPIRED** if the pickup deadline has already passed when the check starts, or by the final yes.
- **Veg feeds need matching:** veg-only needs get only food answered "veg".
- **Badges:** Veg / Non-veg / Covered chips on the kitchen card, the volunteer offer and the dashboard. The spoken alert adds "*Khana veg hai aur dhaka hua hai*".

AUTOMATED TESTS

```
python -m pytest -q tests/test_safety.py
```

TRY IT (FREE)

1. Report food and send **haan**. → "Report confirm ho gayi. Do chhote sawaal. Khana veg hai?" Status: Safety check 1 of 2.
2. **nahi, non-veg hai** → a Non-veg chip; the covered question is asked next.
3. **nahi** → "Kripya khana dhak dijiye, phir haan boliye." A "Not covered yet" chip; the dashboard shows "Waiting for the food to be covered".
4. **haan, dhak diya** → MATCHING, "Safe to send ✓"; the volunteer offer shows the chips and says the food is non-veg and covered.
5. Expired: report food cooked more than 4 hours ago (e.g. **... subah 9 baje bana tha ...** in the evening) and confirm. → no questions; "...pickup samay nikal chuka hai"; the dashboard card is grey and EXPIRED.

LIVE CHECK (SPENDS CREDITS)

- Say short answers aloud (*haan, nahi, athe, illa, ಹೌದು*). These are the words most likely to be misheard.

FEATURE 7

"Kal jaisa": same as yesterday

> A hostel or canteen that reports daily says one phrase instead of dictating everything again.

WHAT WAS ADDED

- **Repeat phrases per language:** *kal jaisa / kal ki tarah / कल जैसा, innale pole / ഇന്നലെത്തെപ്പോലെ, same as yesterday, ninne haage / ನಿನ್ನೆ ಹಾಗೆ*. Changes can come in the same sentence: "*kal jaisa, bas 30 plate*" changes only the quantity.
- **What is copied:** dish, quantity, unit, location and veg, from the kitchen's most recent report that got past confirmation.
- **Cooking time:** never copied; it is asked unless said.
- **Full read-back:** the whole report is always read back, now including veg. "*nahi, non-veg hai*" corrects it, and a new dish clears the copied veg. When veg stays confirmed, the safety check asks only "covered?".
- **Nothing to copy:** the kitchen hears "*Pichhli koi report nahi mili…*", and the next reply is read as a full report.
- **Kitchen page:** a **Last report** card shows what will be copied, with the phrase in the kitchen's language.

AUTOMATED TESTS

```
python -m pytest -q tests/test_repeat.py
```

TRY IT (FREE)

1. Finish one report at Annapurna (any status after the safety check), then press **Start a new report**. → the "Last report" card shows 50 plates · veg biryani · Kakkanad, with a Veg chip.
2. **kal jaisa, bas 30 plate** → "Pichhli report jaisa: 30 plate veg biryani, Kakkanad se. Khana kitne baje bana tha?"
3. Answer with a recent time, e.g. **aath baje** → the full read-back, including "Khana veg hai."
4. **haan** → only one safety question: "Ek chhota sawaal. Khana dhaka hua…?"
5. New dish: **kal jaisa, lekin chicken curry 20 kilo, aath baje bana** → no veg in the read-back; both safety questions are asked.
6. Nothing to copy: on a kitchen with no reports yet (e.g. Temple Road Cafe after a reset), say **same as yesterday** → "I couldn't find an earlier report…"; then say the full report.

LIVE CHECK (SPENDS CREDITS)

- Say each phrase aloud and check the dashboard's "Heard" line. Prisma should spell it the way the word lists expect (*kal jaisa* or कल जैसा, ഇന്നലെത്തെ പോലെ, ನಿನ್ನೆ ಹಾಗೆ).

FEATURE 8

Final integration pass

> Make everything solid for the demo. No new features.

WHAT WAS DONE

- **End-to-end walk in mock mode:** Hinglish and Malayalam, through kitchen → safety → NGO need → volunteer decline → fallback → pickup → delivered → impact card. Both pass.
- **Bugs found and fixed:**
  - the kitchen page sent the first safety answer as a new report;
  - the impact card crashed when the delivery task and a Share click drew it at the same time (now drawn under a lock);
  - the NGO page kept saying "on the way" after delivery;
  - the volunteer card repeated the area name;
  - the dashboard said "Open needs 0" with a need listed;
  - kitchen pages now clear themselves on a demo reset.
- **Demo reset: Reset demo data** opens one need (Karunya, 80 people, about 3 h ahead). For Annapurna's 50 plates, Anjali Menon is asked first and John Mathew second, so the fallback shows when Anjali says no.
- **TTS cache check:** `python -m app.tts_warm` lists the 36 fixed phrases per language and which are cached (free); `--warm` caches the missing ones.
- **Phones:** every page was driven through each demo state at 360 px with no horizontal scroll (emulated phone).
- **Docs:** the README was rebuilt (features, Mermaid architecture and state diagrams, demo script, eval results, honest limits, what's next), and CLAUDE.md now matches the code.

AUTOMATED TESTS

```
python -m pytest -q tests/test_dashboard.py tests/test_tts_warm.py tests/test_cards.py
```

TRY IT: THE FULL DEMO WITH FALLBACK (FREE)

1. Dashboard → **Reset demo data**. → "NGO needs 1": Karunya Children's Home, 80 people.
2. Two volunteer tabs, **Anjali Menon** and **John Mathew**; tap **I'm on duty** in both. Open the NGO page as Karunya.
3. Kitchen (Annapurna): report 50 plates with a recent time → **haan** → **haan veg hai** → **nahi** → **haan dhak diya**.
4. Anjali gets the offer (in Malayalam) → tap **Nahi**. → John gets it a moment later (in English).
5. John: **Haan** → **I've picked it up** → **I've delivered it**. Karunya: **Haan**. → Delivered; impact +50; **Share** works; the timeline shows Anjali's no and John's yes.

BEFORE THE LIVE DEMO

```
$env:MOCK_GNANI="0"; python -m app.tts_warm          # free check: 3 of 144 fixed phrases cached
today
python -m app.tts_warm --warm                         # 141 Timbre calls; only when you decide to
```

CHECKLIST

What still needs a real (non-mock) test

Mock mode proves the logic. These checks need real speech, real phones, or both.

✓	Check	What to look for
☐	Real phones over an HTTPS tunnel	Recording on Android Chrome and iPhone Safari (iPhone records in a different format, which ffmpeg must convert). The offer rings and plays after **I'm on duty**, including with the iPhone on silent.
☐	Prisma per language, real voices	Reports with *pachaas, dhai baje, uchakku rendu manikku*; short replies *haan, nahi, illa, ഇല്ല, ಹೌದು*; "*nahi, non-veg hai*".
☐	Repeat phrases through STT	*kal jaisa, ഇന്നലെത്തെ പോലെ, ನಿನ್ನೆ, ಹಾಗೆ* come back spelled as the word lists expect.
☐	Malayalam and Kannada prompts	Listen with a native speaker: times, numbers, and the "shelter" word used for English names.
☐	Silence check	Real microphones in a noisy kitchen: only silence is rejected, not quiet speech.
☐	Offer timeout	Let an offer ring out (20 s) on a real phone; the next volunteer is called.
☐	Cancel by voice	With a volunteer assigned: the volunteer hears the pickup is cancelled.
☐	Live latency	Read the per-stage averages on the dashboard to quote in the submission.
☐	Telephony mode	`TELEPHONY_MODE=1` with real phone-line audio.
☐	Share on phones	The share sheet and PNG download from the dashboard.
☐	TTS cache and rehearsal	Top up the cache (optional, 141 calls), then one full rehearsal on live credits after a fresh reset. Keep about 1,000 credits in reserve.
☐	Real eval clips	Record `eval/audio/real/ml-01…05` plus safety and repeat replies, run `python eval/run_eval.py --mode all --transcribe`, then paste `eval/RESULTS.md` into the submission.

Mobile layout was checked in an emulated 360 px browser (headless Edge), not on physical phones. All data is synthetic: fictional kitchens, volunteers, shelters and phone numbers around Kochi.